

CircleUp — Day 6

Complete the MVP: Posts, Comments, Footer Branding & Live Demo

Every file below is complete and copy-pasteable.

All tools used are free — no paid API keys or hosting required anywhere in this guide.

Firestore Rules Update

What this is: Extends your Day 5 security rules to cover the two new collections we're adding today — `posts` and `comments`. Still 100% on Firebase's free Spark plan.

Manual steps (Firebase Console)

1. Go to `https://console.firebase.google.com` → your CircleUp project.
2. Left sidebar → **Build** → **Firestore Database** → **Rules** tab.
3. Select all existing text, delete it, paste the rules below.
4. Click **Publish**.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    match /circles/{circleId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update: if request.auth != null;
      allow delete: if false;
    }

    match /posts/{postId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update: if false;
      allow delete: if false;
    }

    match /comments/{commentId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
      allow update: if false;
      allow delete: if false;
    }
  }
}
```

Post & Comment Service

What we're building: The Firestore data layer for posts (inside a circle) and comments (on a post) — same pattern as `circleService.js` from Day 5.

File 1 — `postService.js` NEW

`src/services/postService.js`

```

import {
  collection,
  addDoc,
  doc,
  getDoc,
  onSnapshot,
  query,
  where,
  serverTimestamp,
} from "firebase/firestore";
import { db } from "../firebase";

const postsRef = collection(db, "posts");
const commentsRef = collection(db, "comments");

// Creates a new post inside a circle.
export async function createPost(circleId, authorId, authorName, text) {
  await addDoc(postsRef, {
    circleId,
    authorId,
    authorName,
    text: text.trim(),
    createdAt: serverTimestamp(),
  });
}

// Live-subscribes to all posts belonging to one circle, newest first.
// Sorting happens on-device so we don't need a Firestore composite index
// (keeps this free/simple).
export function subscribeToCirclePosts(circleId, callback) {
  const q = query(postsRef, where("circleId", "=", circleId));
  return onSnapshot(q, (snapshot) => {
    const posts = snapshot.docs.map((d) => ({ id: d.id, ...d.data() }));
    posts.sort((a, b) => (b.createdAt?.seconds || 0) - (a.createdAt?.seconds || 0));
    callback(posts);
  });
}

// Fetches a single post (used on the Post Detail screen).
export async function getPostById(postId) {
  const snap = await getDoc(doc(db, "posts", postId));
  if (!snap.exists()) return null;
  return { id: snap.id, ...snap.data() };
}

// Adds a comment to a post.
export async function createComment(postId, authorId, authorName, text) {
  await addDoc(commentsRef, {
    postId,
    authorId,
    authorName,
    text: text.trim(),
    createdAt: serverTimestamp(),
  });
}

// Live-subscribes to all comments on one post, oldest first (chat-style order).
export function subscribeToPostComments(postId, callback) {
  const q = query(commentsRef, where("postId", "=", postId));
  return onSnapshot(q, (snapshot) => {
    const comments = snapshot.docs.map((d) => ({ id: d.id, ...d.data() }));
    comments.sort((a, b) => (a.createdAt?.seconds || 0) - (b.createdAt?.seconds || 0));
  });
}

```

```
    callback(comments);
  });
}
```

Where it goes: `src/services/postService.js` — brand new file.

DAY 6 · MILESTONE 3

Circle Feed + Post Card

What we're building: A `PostCard` component and a rebuilt Circle Detail screen that shows the join/leave button, a "share an update" box (members only), and a live feed of posts.

File 1 — `PostCard.js` NEW

`src/components/PostCard.js`

```
import React from "react";
import { View, Text, StyleSheet, TouchableOpacity } from "react-native";

export default function PostCard({ post, onPress }) {
  return (
    <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.8}>
      <Text style={styles.author}>{post.authorName}</Text>
      <Text style={styles.text} numberOfLines={4}>
        {post.text}
      </Text>
      <Text style={styles.tap}>View & comment →</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  card: { backgroundColor: "#fff", borderRadius: 12, padding: 14, marginBottom: 10,
    borderWidth: 1, borderColor: "#e5e7eb" },
  author: { fontSize: 13, fontWeight: "700", color: "#2563eb", marginBottom: 4 },
  text: { fontSize: 14, color: "#111827", marginBottom: 6 },
  tap: { fontSize: 11, color: "#9ca3af" },
});
```

Where it goes: `src/components/PostCard.js` — brand new file.

File 2 — `CircleDetailScreen.js` MODIFIED

`src/screens/CircleDetailScreen.js` — replaces the Day 5 version

```

import React, { useEffect, useState } from "react";
import { View, Text, StyleSheet, ActivityIndicator, FlatList } from "react-native";
import Button from "../components/Button";
import Input from "../components/Input";
import PostCard from "../components/PostCard";
import { getCircleById, joinCircle, leaveCircle } from "../services/circleService";
import { createPost, subscribeToCirclePosts } from "../services/postService";
import { useAuth } from "../context/AuthContext";

export default function CircleDetailScreen({ route, navigation }) {
  const { circleId } = route.params;
  const { user } = useAuth();
  const [circle, setCircle] = useState(null);
  const [loading, setLoading] = useState(true);
  const [actionLoading, setActionLoading] = useState(false);
  const [posts, setPosts] = useState([]);
  const [postText, setPostText] = useState("");
  const [posting, setPosting] = useState(false);

  const loadCircle = async () => {
    const data = await getCircleById(circleId);
    setCircle(data);
    setLoading(false);
  };

  useEffect(() => {
    loadCircle();
  }, [circleId]);

  useEffect(() => {
    const unsubscribe = subscribeToCirclePosts(circleId, setPosts);
    return unsubscribe;
  }, [circleId]);

  const isMember = circle?.memberIds?.includes(user.uid);

  const handleJoin = async () => {
    setActionLoading(true);
    await joinCircle(circleId, user.uid);
    await loadCircle();
    setActionLoading(false);
  };

  const handleLeave = async () => {
    setActionLoading(true);
    await leaveCircle(circleId, user.uid);
    await loadCircle();
    setActionLoading(false);
  };

  const handlePost = async () => {
    if (!postText.trim()) return;
    setPosting(true);
    try {
      await createPost(circleId, user.uid, user.displayName || "Member", postText);
      setPostText("");
    } finally {
      setPosting(false);
    }
  };

  if (loading) {

```

```

        return (
            <View style={styles.center}>
                <ActivityIndicator size="large" color="#2563eb" />
            </View>
        );
    }

    if (!circle) {
        return (
            <View style={styles.center}>
                <Text>Circle not found.</Text>
            </View>
        );
    }

    return (
        <FlatList
            style={styles.container}
            contentContainerStyle={{ padding: 24, paddingBottom: 40 }}
            data={isMember ? posts : []}
            keyExtractor={(item) => item.id}
            ListHeaderComponent={
                <View>
                    <Text style={styles.name}>{circle.name}</Text>
                    <Text style={styles.members}>
                        {circle.memberIds?.length || 0} member{(circle.memberIds?.length || 0) === 1 ?
"" : "s"}
                    </Text>
                    <Text style={styles.description}>{circle.description}</Text>

                    {isMember ? (
                        <Button title="Leave Circle" onPress={handleLeave} loading={actionLoading}
variant="secondary" />
                    ) : (
                        <Button title="Join Circle" onPress={handleJoin} loading={actionLoading} />
                    )}

                    {isMember ? (
                        <View style={styles.postBox}>
                            <Input
                                placeholder="Share an update with this circle..."
                                value={postText}
                                onChangeText={setPostText}
                                multiline
                                numberOfLines={3}
                                style={{ height: 80, textAlignVertical: "top" }}
                            />
                            <Button title="Post" onPress={handlePost} loading={posting} />
                        </View>
                    ) : (
                        <Text style={styles.joinPrompt}>Join this circle to see and share updates.</Text>
                    )}

                    {isMember && posts.length > 0 ? <Text style={styles.feedTitle}>Circle Feed</Text> :
null}
                </View>
            }
            renderItem={({ item }) => (
                <PostCard post={item} onPress={() => navigation.navigate("PostDetail", { postId:
item.id })} />
            )}
            ListEmptyComponent={
                isMember ? <Text style={styles.empty}>No posts yet. Be the first to share something!
</Text> : null
        
```

```
    }  
    />  
  );  
}  
  
const styles = StyleSheet.create({  
  container: { flex: 1, backgroundColor: "#fafafa" },  
  center: { flex: 1, justifyContent: "center", alignItems: "center" },  
  name: { fontSize: 26, fontWeight: "800", color: "#111827", marginBottom: 6 },  
  members: { fontSize: 13, color: "#9ca3af", fontWeight: "600", marginBottom: 12 },  
  description: { fontSize: 15, color: "#374151", lineHeight: 22, marginBottom: 20 },  
  postBox: { marginTop: 24 },  
  joinPrompt: { marginTop: 24, fontSize: 13, color: "#9ca3af", textAlign: "center" },  
  feedTitle: { fontSize: 16, fontWeight: "700", color: "#111827", marginTop: 24,  
marginBottom: 10 },  
  empty: { textAlign: "center", color: "#9ca3af", marginTop: 20, fontSize: 13 },  
});
```

Where it goes: `src/screens/CircleDetailScreen.js` — replaces the Day 5 file.

DAY 6 · MILESTONE 4

Post Detail (Comments) + Navigation Wiring

File 1 — `PostDetailScreen.js` NEW

`src/screens/PostDetailScreen.js`


```

import React, { useEffect, useState } from "react";
import {
  View,
  Text,
  StyleSheet,
  FlatList,
  ActivityIndicator,
  KeyboardAvoidingView,
  Platform,
} from "react-native";
import Input from "../components/Input";
import Button from "../components/Button";
import { getPostById, createComment, subscribeToPostComments } from "../services/postService";
import { useAuth } from "../context/AuthContext";

export default function PostDetailScreen({ route }) {
  const { postId } = route.params;
  const { user } = useAuth();
  const [post, setPost] = useState(null);
  const [loading, setLoading] = useState(true);
  const [comments, setComments] = useState([]);
  const [commentText, setCommentText] = useState("");
  const [posting, setPosting] = useState(false);

  useEffect(() => {
    getPostById(postId).then((data) => {
      setPost(data);
      setLoading(false);
    });
  }, [postId]);

  useEffect(() => {
    const unsubscribe = subscribeToPostComments(postId, setComments);
    return unsubscribe;
  }, [postId]);

  const handleComment = async () => {
    if (!commentText.trim()) return;
    setPosting(true);
    try {
      await createComment(postId, user.uid, user.displayName || "Member", commentText);
      setCommentText("");
    } finally {
      setPosting(false);
    }
  };

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#2563eb" />
      </View>
    );
  }

  if (!post) {
    return (
      <View style={styles.center}>
        <Text>Post not found.</Text>
      </View>
    );
  }

```

```

    }

    return (
      <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === "ios" ? "padding" :
undefined}>
        <FlatList
          style={styles.container}
          contentContainerStyle={{ padding: 20, paddingBottom: 40 }}
          data={comments}
          keyExtractor={({item}) => item.id}
          ListHeaderComponent={
            <View>
              <View style={styles.postCard}>
                <Text style={styles.author}>{post.authorName}</Text>
                <Text style={styles.postText}>{post.text}</Text>
              </View>
              <Text style={styles.commentsTitle}>Comments</Text>
            </View>
          }
          renderItem={({ item }) => (
            <View style={styles.commentCard}>
              <Text style={styles.commentAuthor}>{item.authorName}</Text>
              <Text style={styles.commentText}>{item.text}</Text>
            </View>
          )}
          ListEmptyComponent={<Text style={styles.empty}>No comments yet. Start the
conversation!</Text>}
          ListFooterComponent={
            <View style={styles.commentBox}>
              <Input placeholder="Write a comment..." value={commentText}
onChangeText={setCommentText} />
              <Button title="Comment" onPress={handleComment} loading={posting} />
            </View>
          }
        </KeyboardAvoidingView>
      );
    }

    const styles = StyleSheet.create({
      container: { flex: 1, backgroundColor: "#fafafa" },
      center: { flex: 1, justifyContent: "center", alignItems: "center" },
      postCard: { backgroundColor: "#fff", borderRadius: 12, padding: 16, borderWidth: 1,
borderColor: "#e5e7eb", marginBottom: 16 },
      author: { fontSize: 14, fontWeight: "700", color: "#2563eb", marginBottom: 6 },
      postText: { fontSize: 16, color: "#111827", lineHeight: 22 },
      commentsTitle: { fontSize: 16, fontWeight: "700", color: "#111827", marginBottom: 10 },
      commentCard: { backgroundColor: "#fff", borderRadius: 10, padding: 12, marginBottom: 8,
borderWidth: 1, borderColor: "#f3f4f6" },
      commentAuthor: { fontSize: 12, fontWeight: "700", color: "#374151", marginBottom: 2 },
      commentText: { fontSize: 13, color: "#4b5563" },
      empty: { fontSize: 13, color: "#9ca3af", marginBottom: 16 },
      commentBox: { marginTop: 16 },
    });
  }
}

```

Where it goes: `src/screens/PostDetailScreen.js` — brand new file.

File 2 — BrowseStackNavigator.js MODIFIED

src/navigation/BrowseStackNavigator.js – replaces the Day 5 file

```
import React from "react";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import BrowseCirclesScreen from "../screens/BrowseCirclesScreen";
import CircleDetailScreen from "../screens/CircleDetailScreen";
import PostDetailScreen from "../screens/PostDetailScreen";

const Stack = createNativeStackNavigator();

export default function BrowseStackNavigator() {
  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      <Stack.Screen name="BrowseCircles" component={BrowseCirclesScreen} />
      <Stack.Screen name="CircleDetail" component={CircleDetailScreen}
options={{ headerShown: true, title: "" }} />
      <Stack.Screen name="PostDetail" component={PostDetailScreen} options={{ headerShown:
true, title: "Post" }} />
    </Stack.Navigator>
  );
}
```

Where it goes: src/navigation/BrowseStackNavigator.js — replaces the Day 5 file.

File 3 — MyCirclesStackNavigator.js MODIFIED

src/navigation/MyCirclesStackNavigator.js – replaces the Day 5 file

```
import React from "react";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import MyCirclesScreen from "../screens/MyCirclesScreen";
import CircleDetailScreen from "../screens/CircleDetailScreen";
import PostDetailScreen from "../screens/PostDetailScreen";

const Stack = createNativeStackNavigator();

export default function MyCirclesStackNavigator() {
  return (
    <Stack.Navigator screenOptions={{ headerShown: false }}>
      <Stack.Screen name="MyCirclesList" component={MyCirclesScreen} />
      <Stack.Screen name="CircleDetail" component={CircleDetailScreen}
options={{ headerShown: true, title: "" }} />
      <Stack.Screen name="PostDetail" component={PostDetailScreen} options={{ headerShown:
true, title: "Post" }} />
    </Stack.Navigator>
  );
}
```

Where it goes: src/navigation/MyCirclesStackNavigator.js — replaces the Day 5 file.

Footer (Required Branding)

What we're building: A reusable Footer component showing the required challenge attribution text, added to Login, Signup, and Profile so it's visible whether someone is logged in or not — including on the deployed live version.

File 1 — Footer.js NEW

src/components/Footer.js

```
import React from "react";
import { View, Text, StyleSheet } from "react-native";

export default function Footer() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>
        Built with Claude as part of the AB Talks 60-Day Claude AI Challenge.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { paddingVertical: 10, alignItems: "center" },
  text: { fontSize: 11, color: "#9ca3af", textAlign: "center" },
});
```

Where it goes: src/components/Footer.js — brand new file.

File 2 — LoginScreen.js MODIFIED

src/screens/LoginScreen.js — replaces the Day 4 file

```

import React, { useState } from "react";
import {
  View,
  Text,
  StyleSheet,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  TouchableOpacity,
} from "react-native";
import Input from "../components/Input";
import Button from "../components/Button";
import Footer from "../components/Footer";
import { signIn, getAuthErrorMessage } from "../services/authService";

export default function LoginScreen({ navigation }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState("");

  const handleLogin = async () => {
    setErrorMsg("");
    if (!email.trim() || !password) {
      setErrorMsg("Please enter your email and password.");
      return;
    }
    setLoading(true);
    try {
      await signIn(email.trim(), password);
    } catch (error) {
      setErrorMsg(getAuthErrorMessage(error));
    } finally {
      setLoading(false);
    }
  };

  return (
    <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === "ios" ? "padding" :
undefined}>
      <ScrollView contentContainerStyle={styles.container}
keyboardShouldPersistTaps="handled">
        <View>
          <Text style={styles.title}>Welcome back</Text>
          <Text style={styles.subtitle}>Log in to keep up with your circles.</Text>

          <Input
            label="Email"
            placeholder="jane@example.com"
            keyboardType="email-address"
            value={email}
            onChangeText={setEmail}
          />
          <Input
            label="Password"
            placeholder="Your password"
            secureTextEntry
            value={password}
            onChangeText={setPassword}
          />

          {errorMsg ? <Text style={styles.errorBanner}>{errorMsg}</Text> : null}
        </View>
      </ScrollView>
    </KeyboardAvoidingView>
  );
}

```

```

        <Button title="Log In" onPress={handleLogin} loading={loading} />

        <TouchableOpacity style={styles.linkWrapper} onPress={() =>
navigation.navigate("Signup")}>
          <Text style={styles.linkText}>
            New here? <Text style={styles.linkBold}>Create an account</Text>
          </Text>
        </TouchableOpacity>
      </View>

      <Footer />
    </ScrollView>
  </KeyboardAvoidingView>
);
}

const styles = StyleSheet.create({
  container: { flexGrow: 1, justifyContent: "space-between", padding: 24, backgroundColor:
"#fafafa" },
  title: { fontSize: 26, fontWeight: "800", color: "#111827", marginBottom: 6 },
  subtitle: { fontSize: 14, color: "#6b7280", marginBottom: 28 },
  errorBanner: { backgroundColor: "#fef2f2", color: "#b91c1c", padding: 10, borderRadius: 8,
marginBottom: 14, fontSize: 13 },
  linkWrapper: { marginTop: 20, alignItems: "center" },
  linkText: { color: "#6b7280", fontSize: 14 },
  linkBold: { color: "#2563eb", fontWeight: "700" },
});

```

Where it goes: `src/screens/LoginScreen.js` — replaces the Day 4 file.

File 3 — SignupScreen.js MODIFIED

`src/screens/SignupScreen.js` – replaces the Day 4 file

```

import React, { useState } from "react";
import {
  View,
  Text,
  StyleSheet,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  TouchableOpacity,
} from "react-native";
import Input from "../components/Input";
import Button from "../components/Button";
import Footer from "../components/Footer";
import { signUp, getAuthErrorMessage } from "../services/authService";

export default function SignupScreen({ navigation }) {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const [errorMsg, setErrorMsg] = useState("");

  const handleSignup = async () => {
    setErrorMsg("");
    if (!name.trim() || !email.trim() || !password) {
      setErrorMsg("Please fill in all fields.");
      return;
    }
    setLoading(true);
    try {
      await signUp(email.trim(), password, name.trim());
    } catch (error) {
      setErrorMsg(getAuthErrorMessage(error));
    } finally {
      setLoading(false);
    }
  };

  return (
    <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === "ios" ? "padding" :
undefined}>
      <ScrollView contentContainerStyle={styles.container}
keyboardShouldPersistTaps="handled">
        <View>
          <Text style={styles.title}>Create your CircleUp account</Text>
          <Text style={styles.subtitle}>Join circles. Share updates. Meet your people.</Text>

          <Input label="Name" placeholder="Jane Doe" value={name} onChangeText={setName} />
          <Input
            label="Email"
            placeholder="jane@example.com"
            keyboardType="email-address"
            value={email}
            onChangeText={setEmail}
          />
          <Input
            label="Password"
            placeholder="At least 6 characters"
            secureTextEntry
            value={password}
            onChangeText={setPassword}
          />

```

```

      {errorMsg ? <Text style={styles.errorBanner}>{errorMsg}</Text> : null}

      <Button title="Sign Up" onPress={handleSignup} loading={loading} />

      <TouchableOpacity style={styles.linkWrapper} onPress={() =>
navigation.navigate("Login")}>
        <Text style={styles.linkText}>
          Already have an account? <Text style={styles.linkBold}>Log in</Text>
        </Text>
      </TouchableOpacity>
    </View>

    <Footer />
  </ScrollView>
</KeyboardAvoidingView>
);
}

const styles = StyleSheet.create({
  container: { flexGrow: 1, justifyContent: "space-between", padding: 24, backgroundColor:
"#fafafa" },
  title: { fontSize: 26, fontWeight: "800", color: "#111827", marginBottom: 6 },
  subtitle: { fontSize: 14, color: "#6b7280", marginBottom: 28 },
  errorBanner: { backgroundColor: "#fef2f2", color: "#b91c1c", padding: 10, borderRadius: 8,
marginBottom: 14, fontSize: 13 },
  linkWrapper: { marginTop: 20, alignItems: "center" },
  linkText: { color: "#6b7280", fontSize: 14 },
  linkBold: { color: "#2563eb", fontWeight: "700" },
});

```

Where it goes: `src/screens/SignupScreen.js` — replaces the Day 4 file.

File 4 — HomeScreen.js MODIFIED

`src/screens/HomeScreen.js` — replaces the Day 5 file


```
import React from "react";
import { View, Text, StyleSheet } from "react-native";
import Button from "../components/Button";
import Footer from "../components/Footer";
import { useAuth } from "../context/AuthContext";
import { logOut } from "../services/authService";

export default function HomeScreen() {
  const { user } = useAuth();

  return (
    <View style={styles.container}>
      <View style={styles.top}>
        <Text style={styles.title}>{user?.displayName || "Your Profile"}</Text>
        <Text style={styles.subtitle}>{user?.email}</Text>
        <Button title="Log Out" onPress={logOut} variant="secondary" />
      </View>
      <Footer />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "space-between", alignItems: "center", padding: 24,
backgroundColor: "#fafafa" },
  top: { flex: 1, justifyContent: "center", alignItems: "center", width: "100%" },
  title: { fontSize: 22, fontWeight: "800", color: "#111827", marginBottom: 4 },
  subtitle: { fontSize: 14, color: "#6b7280", marginBottom: 24 },
});
```

Where it goes: `src/screens/HomeScreen.js` — replaces the Day 5 file.

Free tools confirmed for today: Firebase Firestore (Spark/free plan) for posts and comments. No new packages needed today, no paid API keys anywhere.

Run & Test

```
npx expo start
```

Scan the QR code with Expo Go (or press `a` / `i` for an emulator).

Full Day 6 Test Checklist

1. Firestore rules published (posts/comments added).
2. Create the 4 new files: `postService.js` , `PostCard.js` , `PostDetailScreen.js` , `Footer.js` .
3. Replace the 6 modified files: `CircleDetailScreen.js` , `BrowseStackNavigator.js` , `MyCirclesStackNavigator.js` , `LoginScreen.js` , `SignupScreen.js` , `HomeScreen.js` .
4. Run `npx expo start` — confirm the footer text appears at the bottom of the Login and Profile screens.
5. Open a circle you've joined → write an update → tap Post → confirm it appears instantly under "Circle Feed."
6. Tap a post → write a comment → tap Comment → confirm it appears immediately.
7. Confirm signup, login, logout, join/leave (Days 4–5) all still work.

Deployment Note

CircleUp is a mobile app (React Native/Expo), not a website — so Vercel/Netlify/Render don't apply here. The free way to get a shareable live demo is **Expo's EAS Update**, which produces a link + QR code that opens the real app in Expo Go on anyone's phone. Entirely free on Expo's free tier. Once you've confirmed the checklist above works, send a screenshot and the next step will walk through EAS Update deployment.

Commit & Push to GitHub

In Command Prompt, inside your project folder:

```
git add .
git commit -m "Day 6: Add posts, comments, and required footer branding to complete MVP"
git push origin main
```

Replace `main` with your actual branch name if different (check with `git branch`).

Day 6 Summary

Completed Today	Still Needs Polishing
Firestore rules extended for posts/comments Post service (create/read/live feed) Comment service (create/read/live list) Circle feed integrated into Circle Detail Post Detail screen with comments Required footer added (Login, Signup, Profile)	Live shareable demo via EAS Update (pending test confirmation) Optional: loading/error polish, empty-state illustrations Optional: profile pictures, richer post media

Next: Confirm the test checklist, then deploy via EAS Update for a shareable live demo link.